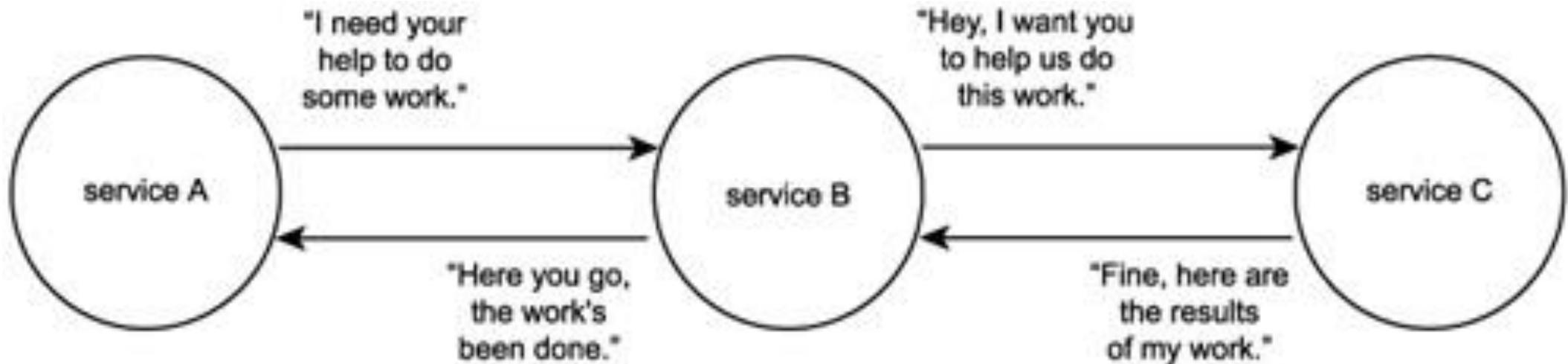


Web Services Orchestration and Choreography

Prof. Prashant M. Jadav
Associate Professor,
Computer Engineering Department
Faculty of Technology,
Dharmsinh Desai University, Nadiad

Service Activity

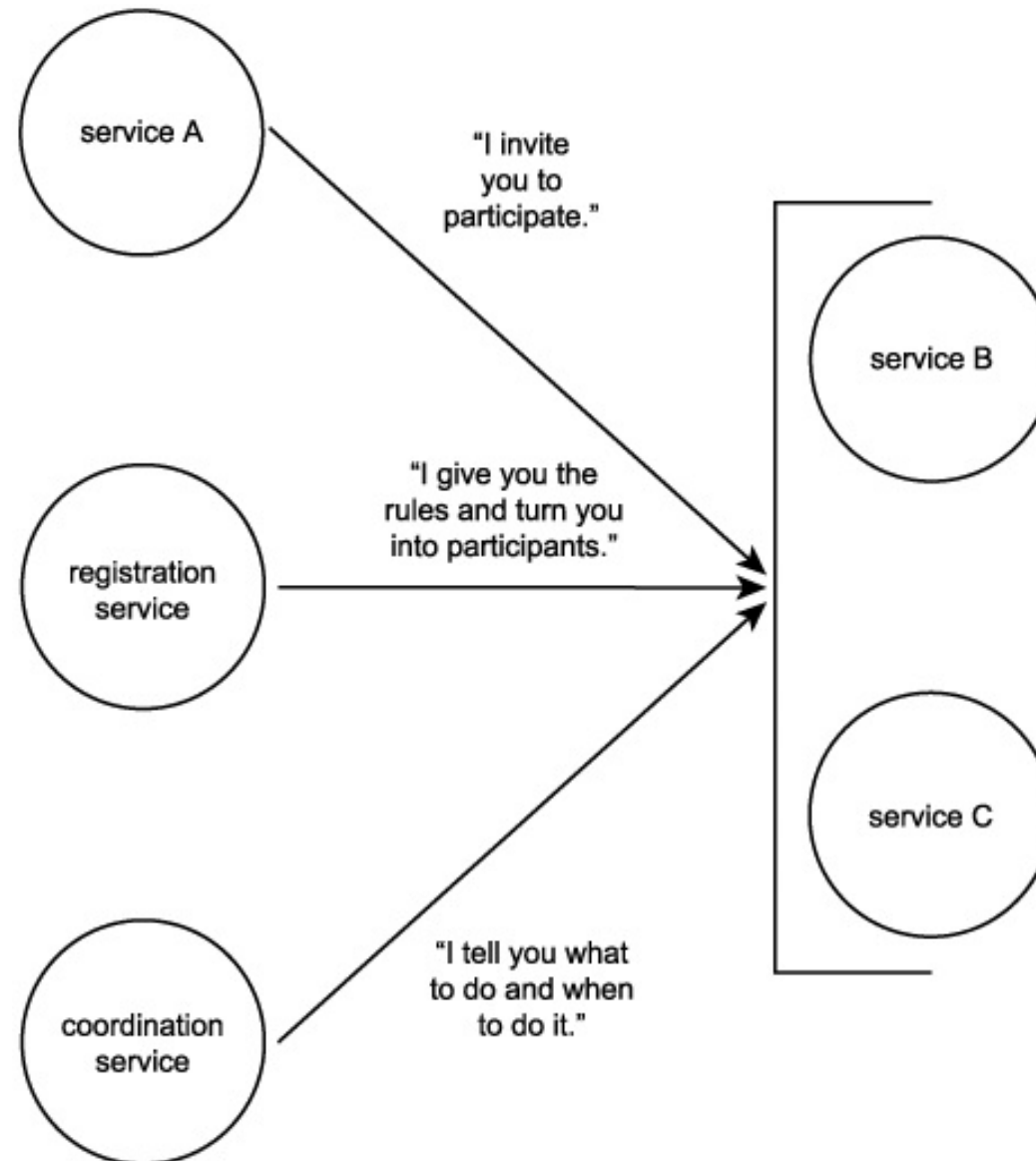
- The interaction of a group of services working together to complete a task is referred to as a **service activity**



Service Coordination

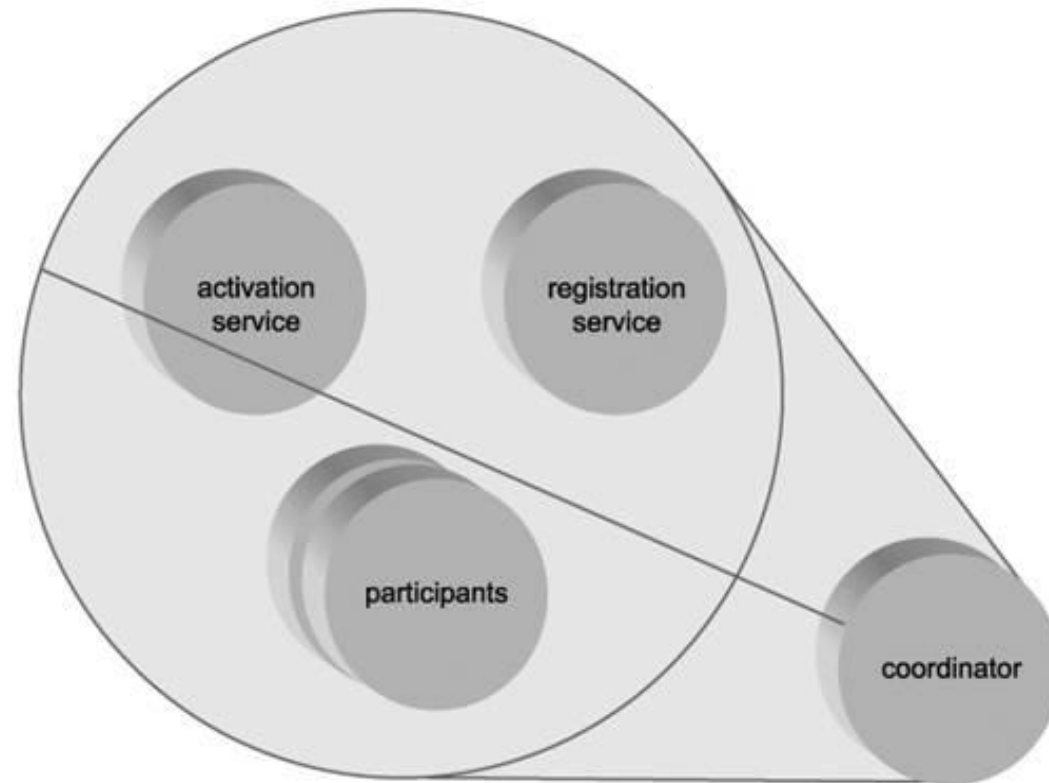
- The complexity of an activity can relate to a number of factors:
 - the amount of services that participate in the activity
 - the duration of the activity
 - the frequency with which the activity changes
 - whether multiple instances of the activity can concurrently exist
- A Coordination framework
 - maintains, manages, preserves, updates, and distributes
- the context information in complex activities

Service Coordination



WS – Coordination

- **WS-Coordination** is a framework that introduces a generic service which controls a composition of three other services that each play a specific part in the management of context data



WS – Coordination

- **Activation service** Responsible for the creation of a new context and for associating this context to a particular activity
- **Registration service** Allows participating services to use context information received from the activation service to register for a supported context protocol
- **Protocol-specific services** These services represent the protocols supported by the coordinator's coordination type
- **Coordinator** The controller service of this composition

Coordination Type

- **Coordination type** specifies the nature and underlying logic of an activity for which context information is being managed
- **Coordination Types**
 - Custom
 - WS – AtomicTransaction
 - WS – BusinessActivity

Coordination Protocols

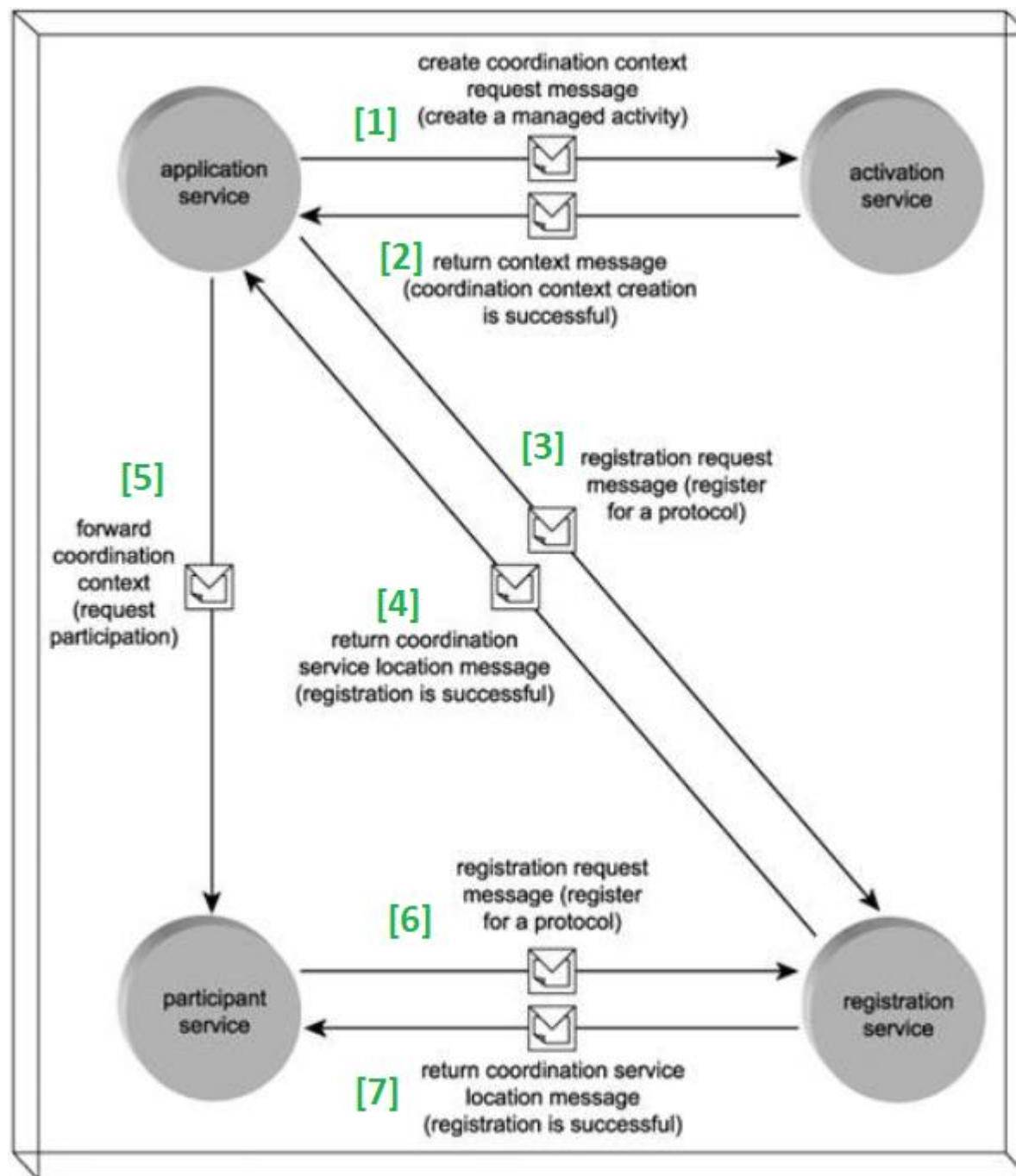
- Coordination type extensions provide a set of coordination protocols
- It represent
 - unique variations of coordination types and
 - consist of a collection of specific behaviors and rules
- These rules are imposed on activities and must be followed by all registered participants

Coordination Context

- A **context** created by the activation service which contains information about the **activity** and **supplementary data**
- The context include:
 - a **unique identifier**
 - an **expiration value**
 - **coordination type**
- A service **request** the coordination context from the **activation service**
- It uses this context information to **register for** one or more **coordination protocols** (e.g. **WS-AtomicTransaction**, **WS-Business Activity**)
- Such service is considered a **participant** in the coordinated activity

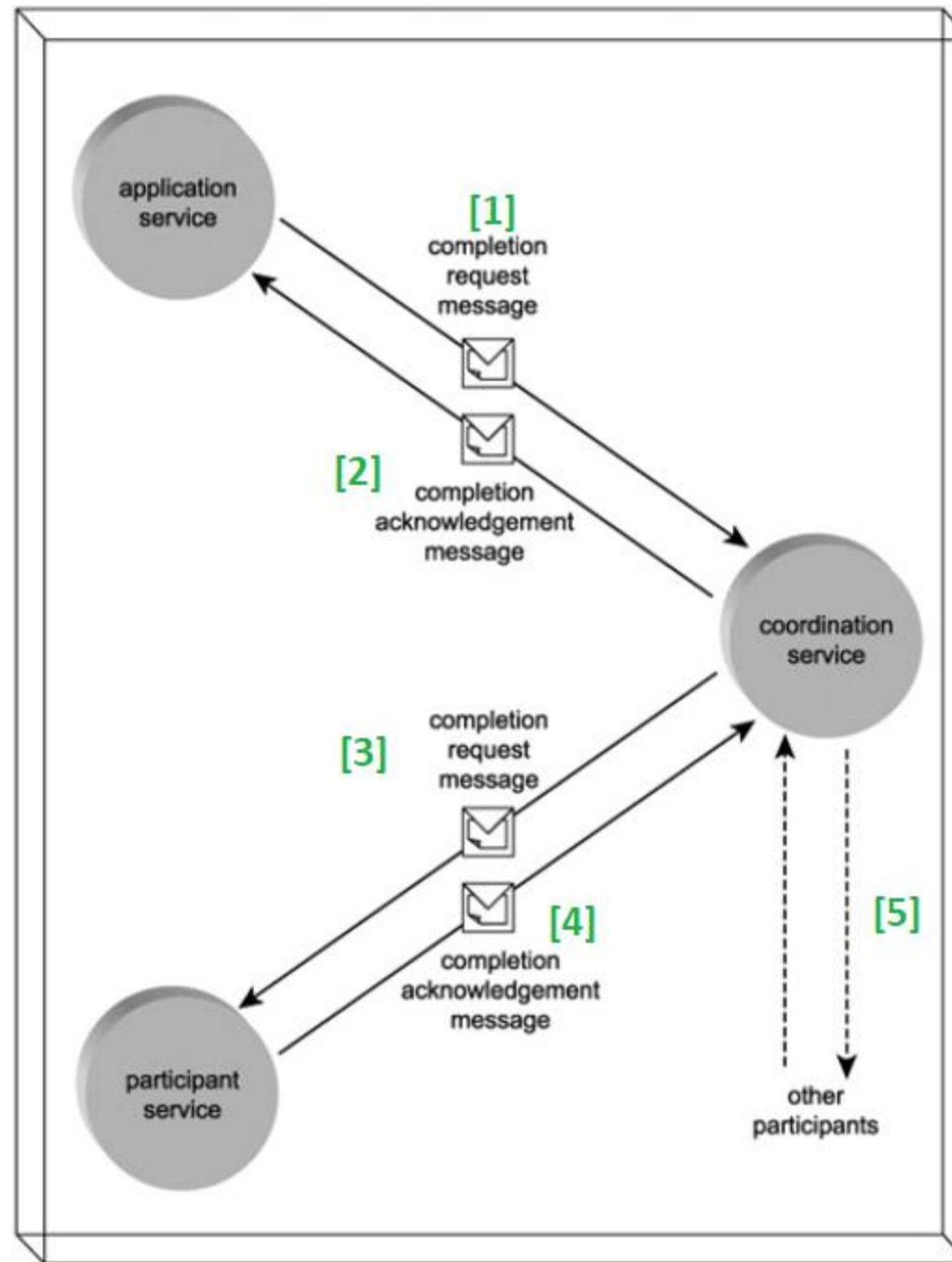
The Activation and Registration Process

- The Coordination service composition is instantiated when an Application service contacts the Activation service
- It requests a **CreateCoordinationContext message** to Activation service, and asks to generate a set of new context data
- The **ReturnContext message** is passed back by Activation service (which also sends the address of the participant to Coordination Service)
- The Application service now can **invite other services** to **participate** in the coordination along with the context information



The Completion Process

- The **Application service** can request that a coordination be completed by issuing a **completion request message** to the coordination service
- The **Coordinator**, in turn, then issues its own **completion request messages** to all coordination participants
- Each **participant service** responds with a **completion acknowledgement message**



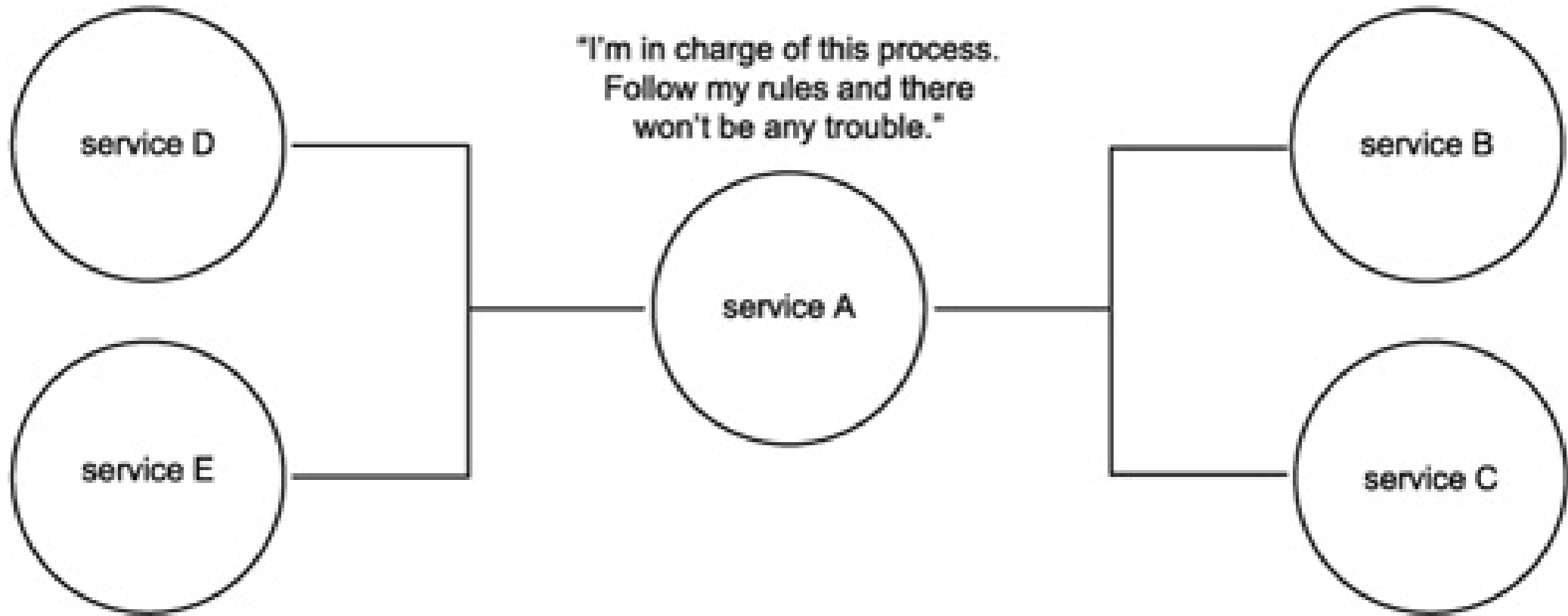
Orchestration

- In orchestration system, a centrally controlled set of workflow logic facilitates interoperability between two or more different applications
- A common implementation is the hub-and-spoke model that allows multiple external participants to interface with a central orchestration engine
- One of the driving requirements is to accommodate the merging of large business processes
- With orchestration, different processes can be connected without having to redevelop the solutions that originally automated the processes individually
- Orchestration bridges this gap by introducing new workflow logic
- The use of orchestration can significantly reduce the complexity of solution environments

Orchestration

- Workflow logic is abstracted and more easily maintained than when embedded within individual solution components
- Orchestration can represent and express business logic in a standardized, services-based venue
- It provides an extremely attractive means of housing and controlling the logic representing the process being automated
- It leverages the intrinsic interoperability sought by service designs by providing potential integration endpoints into processes
- A key aspect is that orchestrations themselves exist as services
- Orchestration logic standardizes process representation across an organization
- Addresses the goal of enterprise federation and promotes SOA

Orchestration

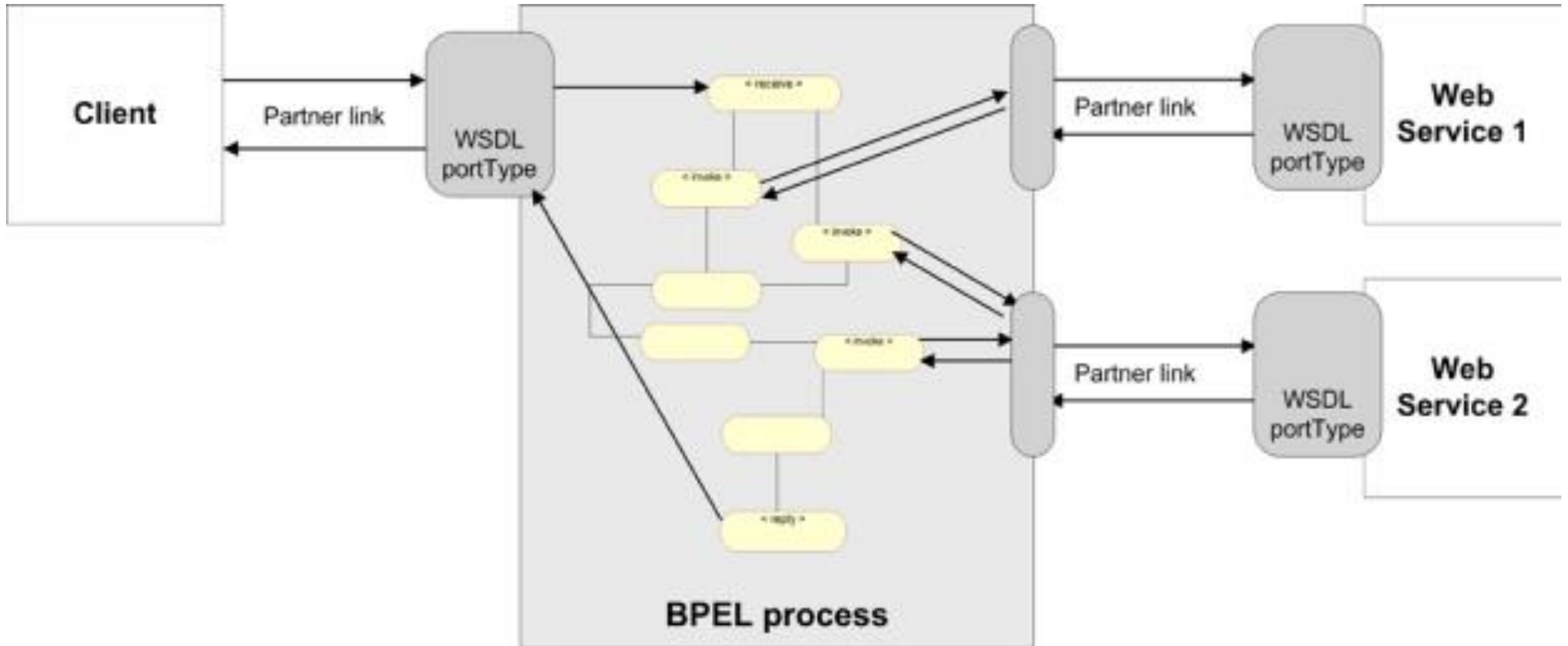


Orchestration



[Image by macrovector on Freepik](#)

Orchestration



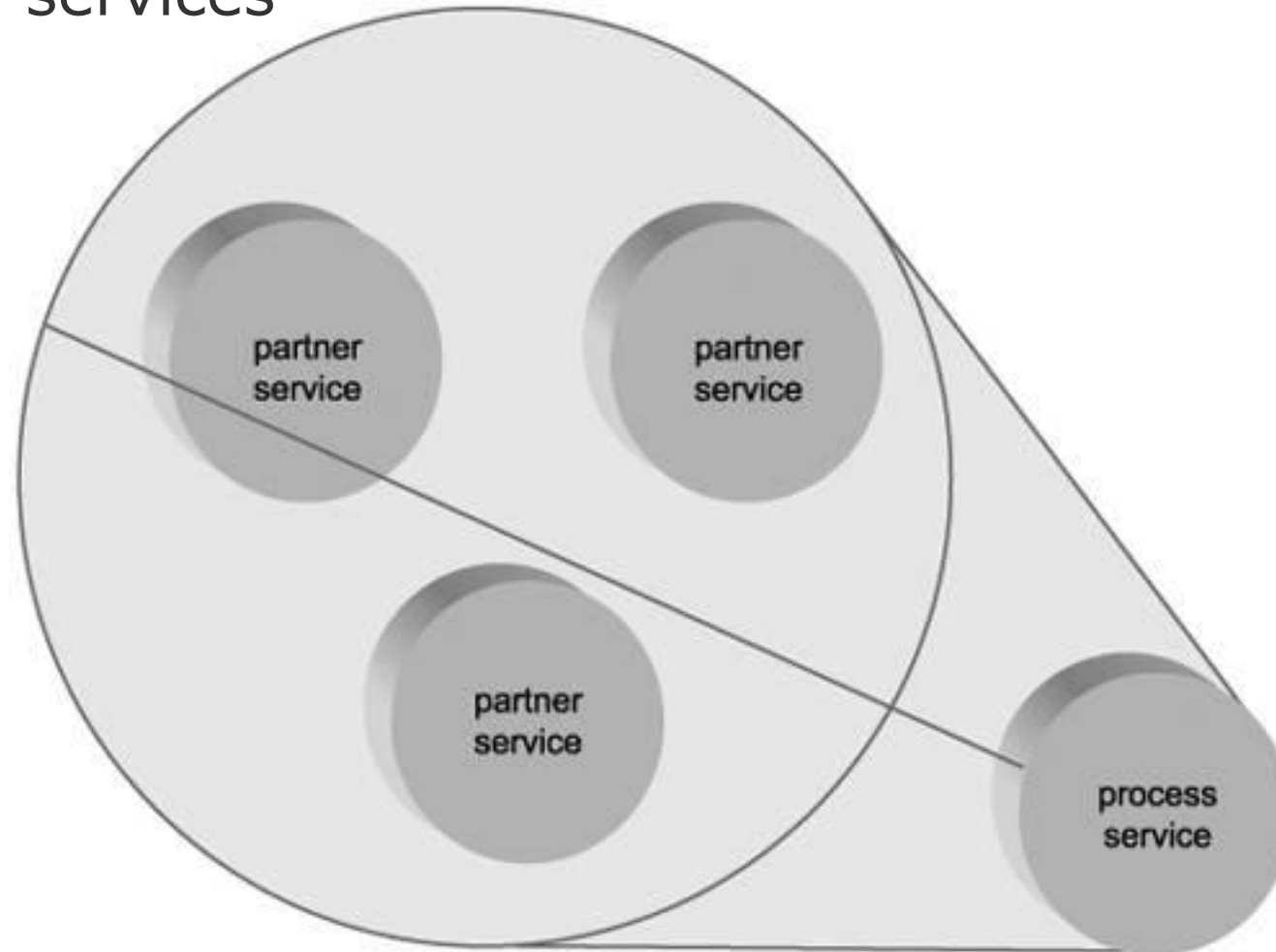
<https://dzone.com/refcardz/core-ws-bpel-business-process>

WS-BPEL (Business Process Execution Language)

- The workflow logic comprises an orchestration and consist of
 - Business rules,
 - Conditions, and
 - Events
- Collectively, these parts of an orchestration establish a *business protocol* that defines how participants can interoperate to achieve the completion of a business task
- The details of the workflow logic encapsulated and expressed by an orchestration are contained within a *process definition*

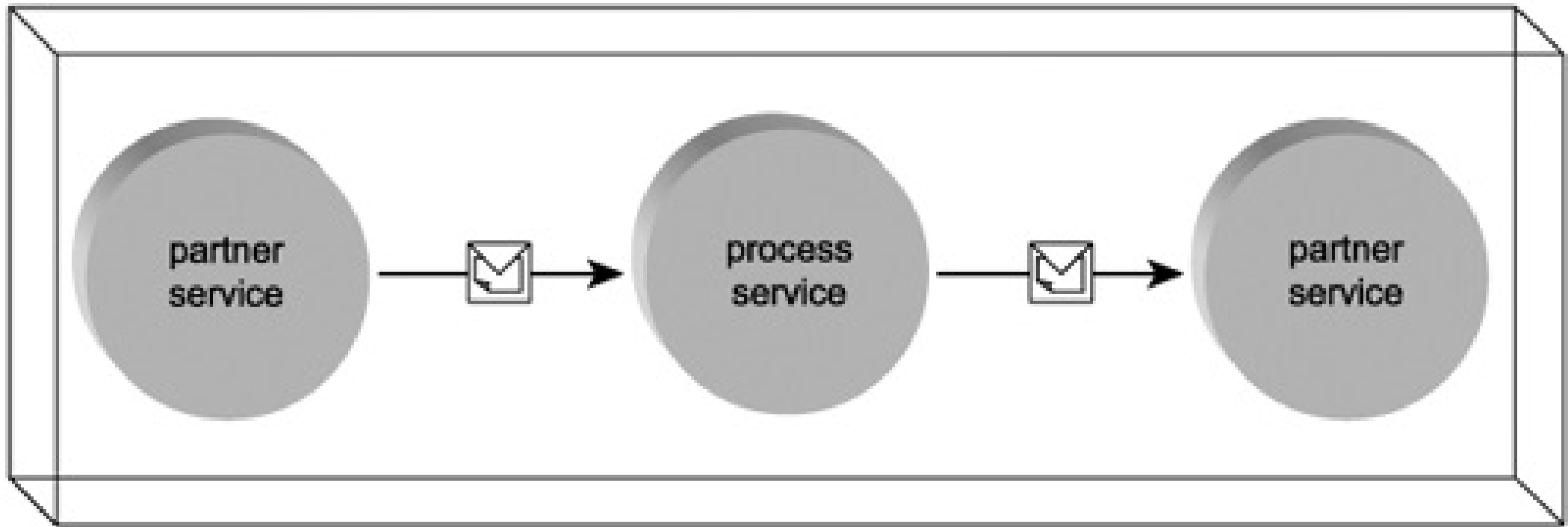
Process Services and Partner Services

- A **Process service** coordinates and exposes functionality from multiple partner services



Process Services and Partner Services

- The process service can be invoked by an external partner service, or it can invoke other partner services



Basic Activity Services and Structured Services

- WS-BPEL breaks down **workflow logic** into a series of predefined **Basic activities**
 - **receive, invoke, reply, throw, wait**
- Which represent fundamental **workflow actions** which can be assembled using the logic supplied by **structured activities**
 - **sequence, switch, while, flow, pick**

Sequences, Flows and Links

- Basic and structured activities can be organized so that the order in which they execute is predefined
- A sequence aligns groups of related activities into a list that determines a sequential execution order
- Sequences are especially useful when one piece of application logic is dependent on the outcome of another

Sequences, Flows and Links

- **Flows** contain **groups of related activities**, but they introduce different execution requirements
- Pieces of **application logic** can **execute concurrently within a flow**
- One set of **activities need not wait before another finishes**
- The **flow does not finish** until all encapsulated activities **have completed**
- This **ensures** a **form of synchronization** among application logic residing in individual flows

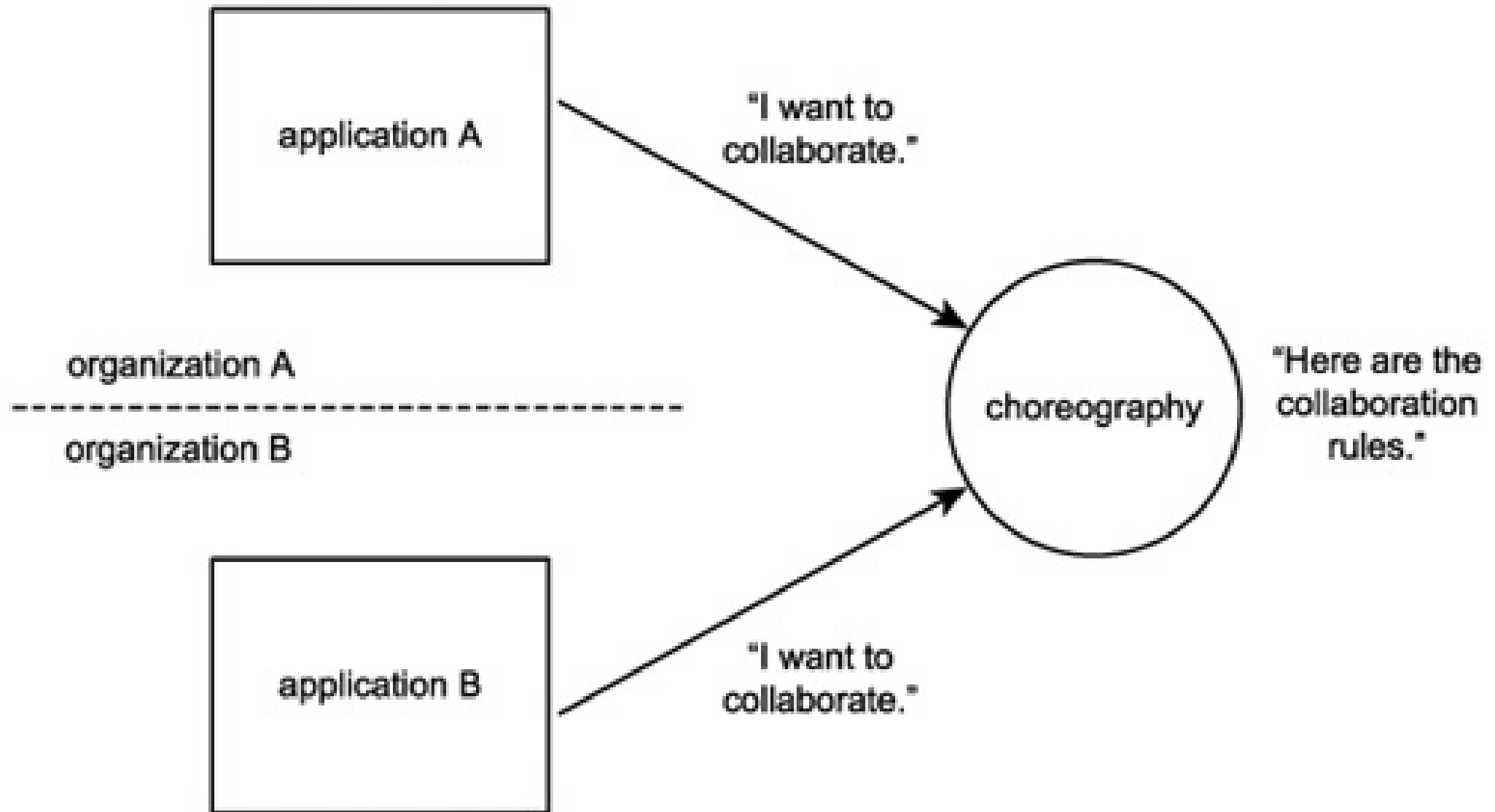
Sequences, Flows and Links

- Links are used to establish formal dependencies between activities that are part of flows
- Before any linked activity can begin, requirements contained within any incoming links first must be satisfied
- Rules provided by links are also referred to as synchronization dependencies

Choreography

- Multiple services from different organizations or within the organization need to work together to achieve a common goal
- The Web Services Choreography Description Language (WS-CDL) is a specifications that attempts to organize information exchange between multiple organizations (or even multiple applications within organizations)
- It emphasis on public collaboration

Choreography



Choreography



[Image by master1305 on Freepik](#)

Collaboration

- An important characteristic of choreographies is that they are intended for public message exchanges
- The goal is to establish a kind of organized collaboration between services representing different service entities
- No one entity necessarily controls the collaboration logic
- It provides the potential for establishing universal interoperability patterns for common inter-organization business tasks

Roles and Participants

- A Web service assumes one of a number of predefined roles
- This establishes what the service does and what the service can do within the context of a particular business task
- Roles can be bound to WSDL definitions
- Related services are grouped and categorized as participants

Relationships and Channels

- Every **action** is a series of message exchanges between two services
- Each exchange between two roles is defined individually as a **relationship**
- **Channels** defines the characteristics of the message exchange between two specific roles
- To facilitate more complex exchanges involving multiple participants, **channel information can be passed around in a message**
- This allows one service to send another the information required for it to be communicated with by other services
- This feature of the WS-CDL specification, fosters dynamic discovery and increases the number of potential participants within large-scale collaborative tasks

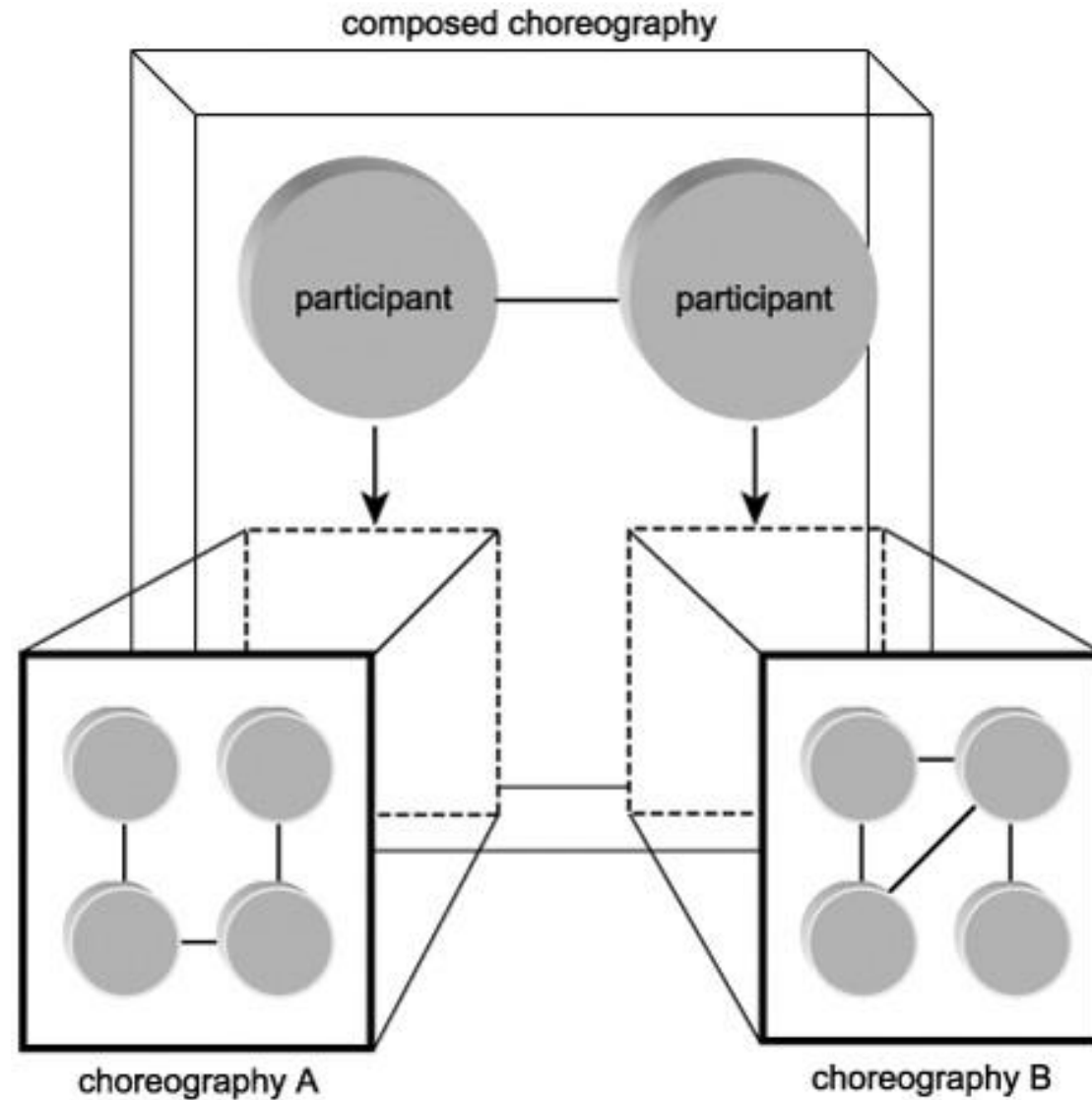
Interactions and Work Units

- The **actual logic** behind a message exchange is **encapsulated within an interaction**
- The **completion of an interaction** represents **actual progress** within a choreography
- **Work units impose rules and constraints** that must be adhered to for an interaction to successfully complete

Reusability, Composability and Modularity

- Each choreography can be designed in a *reusable* manner
 - allowing it to be applied to different business tasks
- Using an import facility, a choreography can be assembled from independent *modules*
- These modules can represent distinct sub-tasks
 - which can be reused by different parent choreographies

Reusability, Composability and Modularity



References

- “Service-Oriented Architecture: Concepts, Technology, and Design” by Thomas Erl, Pearson Education